

AWS–Azure Site-to-Site VPN Lab

1. What you are building

You will create:

- An AWS VPC with an EC2 instance.
- An Azure VNet with an Ubuntu VM.
- An AWS Virtual Private Gateway attached to the VPC.
- An Azure VPN Gateway deployed in GatewaySubnet.
- A VPN connection between the two cloud gateways.
- Routes and firewall rules allowing private communication.
- Terraform configuration to provision and connect these resources.

The gateways use public IP addresses to establish the encrypted tunnel. The EC2 instance and Azure VM communicate using private IP addresses inside that tunnel.

This VPN travels over the public internet; it does not create a dedicated physical connection.

2. Understand the components

Component	Purpose
AWS VPC	Your private network in AWS
Azure VNet	Your private network in Azure
EC2 instance	AWS server used to test connectivity
Azure VM	Azure server used to test connectivity
AWS Virtual Private Gateway (VGW)	AWS endpoint that terminates the VPN
AWS Customer Gateway (CGW)	AWS configuration representing the Azure gateway
Azure VPN Gateway	Azure endpoint that terminates the VPN
Azure Local Network Gateway (LNG)	Azure configuration representing the AWS endpoint and network
VPN connection	Configuration joining the two gateways
Pre-shared key (PSK)	Secret used to authenticate the VPN peers
Route table	Determines where network traffic goes
Security Group / NSG	Controls permitted traffic to your servers
Terraform	Creates and manages infrastructure using code

The Customer Gateway and Local Network Gateway are configuration objects. You do not deploy an extra VM for either one.

3. Plan the IP addresses

Use networks that do not overlap.

Resource	Example
AWS region	Mumbai — ap-south-1
AWS VPC	10.10.0.0/16
AWS workload subnet	10.10.1.0/24
EC2 private IP	10.10.1.10
Azure region	Central India
Azure VNet	10.20.0.0/16
Azure workload subnet	10.20.1.0/24
Azure VM private IP	10.20.1.10
Azure GatewaySubnet	10.20.255.0/27

These are example addresses. Configure them explicitly if you want to use the exact commands later in this guide.

Do not assign **10.10.0.0/16** to both clouds. Overlapping address spaces make ordinary routing ambiguous.

For this first lab:

- AWS VPN routing: **Static**
- Azure VPN type: **Route-based**
- Azure BGP: **Disabled**
- Azure gateway mode: **Active-standby**
- Initially configure **one** AWS tunnel

Tip: “Route-based VPN” and “static routing” are different settings. A route-based Azure gateway can connect to an AWS VPN using static network routes. AWS supports both static routing and dynamic routing through BGP.

4. Prepare your accounts and tools

You need:

1. An AWS account with permission to manage VPC, EC2, VPN, and related resources.
2. An Azure subscription with permission to create networking and VM resources.
3. Terraform installed.
4. AWS CLI installed.
5. Azure CLI installed.
6. Git and a code editor.

7. An SSH client such as MobaXterm.

Check your tools:

```
terraform version
aws --version
az version
git --version
```

Authenticate to AWS (use an AWS SSO profile instead if your account uses IAM Identity Center):

```
aws configure
aws sts get-caller-identity
```

Authenticate to Azure:

```
az login
az account list --output table
az account set --subscription "<your-subscription-id>"
az account show --output table
```

Cost warning: Set a budget before deployment. VPN gateways, AWS VPN connections, VMs, public IPs, and applicable data transfer can incur charges. Stopping the VMs does not remove gateway charges.

5. Follow this creation order

The order matters because the clouds need information from each other.

1. Create the AWS VPC and Azure VNet.
2. Create the Azure VPN Gateway and obtain its public IP.
3. Create the AWS Virtual Private Gateway.
4. Create the AWS Customer Gateway using the Azure gateway public IP.
5. Create the AWS VPN connection.
6. Obtain the AWS tunnel public IP and its pre-shared key.
7. Create the Azure Local Network Gateway using that AWS tunnel IP.
8. Create the Azure VPN connection using the matching key.
9. Configure workload routes and firewall rules.
10. Test private connectivity.

This order also gives Terraform a dependency path without creating a circular reference.

6. Create the AWS network

In **AWS Console** → **VPC**:

1. Create VPC `aws-multicloud-vpc`.
2. Set IPv4 CIDR to `10.10.0.0/16`.
3. Create subnet `aws-workload-subnet`.
4. Set subnet CIDR to `10.10.1.0/24`.
5. Create a route table.
6. Associate that route table with the workload subnet.

The route table initially contains:

Destination	Target
10.10.0.0/16	Local

Choose how you will manage the EC2 instance:

- For a simple lab, use a public IP with SSH restricted to your own public IP.
- For a private instance, prepare Systems Manager access through suitable endpoints or outbound connectivity.

A public IP used for administration does not prevent private application traffic from using the VPN.

7. Create the Azure network

In **Azure Portal** → **Virtual networks**:

1. Create resource group `rg-aws-azure-vpn-lab`.
2. Create VNet `azure-multicloud-vnet`.
3. Set address space to `10.20.0.0/16`.
4. Create workload subnet `azure-workload-subnet`, `10.20.1.0/24`.
5. Create subnet exactly named `GatewaySubnet`, `10.20.255.0/27`.

Important: GatewaySubnet is reserved for the managed VPN gateway. Do not place your application VM there or attach a workload NSG to it.

8. Create the Azure VPN Gateway

In **Azure Portal** → **Virtual network gateways** → **Create**:

Setting	Value
Name	<code>azure-vpn-gateway</code>
Region	Same as your VNet
Gateway type	VPN
VPN type	Route-based
Virtual network	<code>azure-multicloud-vnet</code>
SKU	A supported non-Basic SKU, such as an available VpnGw1 tier
Active-active	Disabled for this first lab
BGP	Disabled
Public IP	Create a compatible Standard, static public IP

Check the currently available SKU options in your subscription before creating the gateway.

Wait until deployment finishes, then record the gateway public IP:

`AZURE_GATEWAY_PUBLIC_IP = <actual Azure gateway public IP>`

Microsoft’s portal tutorial covers the VNet, gateway, local network gateway, connection, and verification sequence. Gateway deployment makes this a longer-running setup.

9. Create the AWS Virtual Private Gateway

In **AWS VPC → Virtual private gateways**:

1. Create `aws-vgw`.
2. Use the default Amazon-side ASN for this static-routing lab.
3. Attach the gateway to `aws-multicloud-vpc`.

An ASN identifies a routing system when using BGP. It does not control route exchange in this static-routing setup.

10. Create the AWS Customer Gateway

In **AWS VPC → Customer gateways**:

1. Create `azure-cgw`.
2. Enter the Azure VPN Gateway public IP.
3. Supply a valid ASN if the form requires it; `65515` is a common Azure ASN.
4. Create the resource.

Important: The Customer Gateway IP must be the Azure gateway endpoint, not the Azure VM public IP.

For a later BGP implementation, this ASN must match the actual Azure gateway BGP ASN.

11. Create the AWS Site-to-Site VPN connection

In **AWS VPC → Site-to-Site VPN connections → Create**:

Setting	Value
Name	<code>aws-to-azure-vpn</code>
Target gateway	Virtual Private Gateway
Virtual Private Gateway	<code>aws-vgw</code>
Customer Gateway	<code>azure-cgw</code>
Routing	Static
Static IP prefix	<code>10.20.0.0/16</code>

The static prefix represents the network reachable on the Azure side.

AWS creates two tunnels, each with a different public endpoint. Record Tunnel 1’s endpoint and key. AWS recommends configuring both tunnels for redundancy; the first version of this guide deliberately establishes one working tunnel before expanding the design.

```
AWS_TUNNEL_1_PUBLIC_IP = <actual AWS tunnel 1 outside IP>
TUNNEL_1_PSK           = <actual tunnel 1 pre-shared key>
```

You can download the VPN configuration from AWS. Treat that file as sensitive because it can contain keys.

12. Match the IPsec settings

Both ends need compatible negotiation settings. A suitable lab policy to configure explicitly is:

Setting	AWS	Azure
IKE version	IKEv2	IKEv2
Phase 1 encryption	AES256	AES256
Phase 1 integrity	SHA2-256	SHA256
Phase 1 DH group	14	DHGroup14
Phase 2 encryption	AES256	AES256
Phase 2 integrity	SHA2-256	SHA256
Phase 2 PFS group	14	PFS2048
Phase 2 lifetime	3600 seconds	3600 seconds
Authentication	Tunnel 1 PSK	Same Tunnel 1 PSK

On AWS, a Phase 1 lifetime of 28800 seconds is supported. On Azure, configure the supported custom policy fields rather than expecting every AWS setting to have an identical portal field.

AWS and Azure use different names for some equivalent algorithms. Verify the selected SKU supports your Azure custom policy. AWS documents its allowed tunnel options, and Microsoft documents its IPsec/IKE policy configuration.

Warning: Do not mix Tunnel 1's endpoint with Tunnel 2's key.

13. Create the Azure Local Network Gateway

In Azure Portal → Local network gateways → Create:

Setting	Value
Name	<code>aws-tunnel1-lng</code>
Endpoint	IP address
IP address	AWS Tunnel 1 public IP
Address space	<code>10.10.0.0/16</code>
BGP	Disabled

The address space here is the AWS network, because AWS is the remote network from Azure's perspective.

14. Create the Azure VPN connection

Open `azure-vpn-gateway` → **Connections** → **Add**, then configure:

Setting	Value
Name	<code>azure-to-aws-tunnel1</code>
Connection type	Site-to-site / IPsec
Virtual network gateway	<code>azure-vpn-gateway</code>
Local network gateway	<code>aws-tunnel1-lng</code>
Shared key	AWS Tunnel 1 PSK
IKE protocol	IKEv2
BGP	Disabled
IPsec/IKE policy	Matching custom policy

For the route-based configuration, leave policy-based traffic selectors disabled unless you intentionally implement a design that needs them.

Generate traffic after creation and check:

- Azure connection status: **Connected**
- AWS Tunnel 1 status: **UP**

Tunnel status is the first check. It does not yet prove that the servers can communicate.

15. Configure the AWS route table

Add this route to the route table associated with your EC2 subnet:

Destination	Target
<code>10.20.0.0/16</code>	<code>aws-vgw</code>

Your table now includes:

Destination	Target
<code>10.10.0.0/16</code>	Local
<code>10.20.0.0/16</code>	Virtual Private Gateway
<code>0.0.0.0/0</code>	Internet Gateway, only if using a public administration subnet

There are two separate AWS routing tasks:

1. Add Azure's CIDR to the VPN connection's static routes.
2. Ensure the EC2 subnet route table sends Azure-bound traffic to the VGW.

You can use route propagation instead of a manually added remote route, but choose one clear approach for this lab.

16. Verify Azure routing

The Azure Local Network Gateway contains **10.10.0.0/16**.

With the connection configured, check the Azure VM NIC's effective routes. The AWS prefix should be reachable through the virtual network gateway.

For this straightforward setup, start without a custom Azure route table. If you attach a route table:

- Keep gateway route propagation enabled.
- Check that no conflicting route sends **10.10.0.0/16** elsewhere.
- Avoid overriding gateway infrastructure routes.

A route tells traffic where to go. It does not grant permission through an NSG or Security Group.

17. Configure the AWS Security Group

Attach a Security Group to EC2. Example inbound rules:

Purpose	Protocol / port	Source
SSH from Azure VM	TCP 22	10.20.1.10/32
Web test from Azure VM	TCP 80	10.20.1.10/32
Ping from Azure VM	ICMP Echo Request	10.20.1.10/32
Administrative SSH	TCP 22	Your own public IP /32, if required

Allow outbound traffic needed for your tests. For the lab, the default outbound rule can simplify initial troubleshooting; tighten it afterward.

You do not need UDP 500 or 4500 on the EC2 Security Group. The managed gateways terminate IPsec; the workload receives the inner application traffic.

If you use custom Network ACLs, permit the necessary traffic and return traffic. NACLs are stateless.

18. Configure the Azure NSG

Attach the NSG to the workload subnet or VM NIC. Example inbound rules:

Priority	Purpose	Protocol / port	Source
100	SSH from EC2	TCP 22	10.10.1.10/32
110	Web test from EC2	TCP 80	10.10.1.10/32
120	Ping from EC2	ICMP	10.10.1.10/32
130	Administrative SSH	TCP 22	Your own public IP /32, if required

Check effective security rules, including existing default rules. If both NIC and subnet NSGs are attached, traffic must pass both.

Also check the Linux host firewall:

```
sudo ufw status
```

If UFW is enabled, add appropriately scoped rules for the remote server rather than disabling the firewall.

19. Create the two test servers

AWS EC2

- Ubuntu
- Workload subnet: **10.10.1.0/24**
- Example private IP: **10.10.1.10**
- Your AWS test Security Group

Azure VM

- Ubuntu
- Workload subnet: **10.20.1.0/24**
- Example private IP: **10.20.1.10**
- Your Azure test NSG

Use SSH keys for access. For a simple lab, public IPs may be used for administration. Use private IPs for every cross-cloud test.

On both servers, install Nginx:

```
sudo apt update
sudo apt install -y nginx curl
sudo systemctl enable --now nginx
```

This installation requires internet access or another available package source. Private-only servers need a suitable outbound path.

On EC2:

```
echo "Response from AWS EC2 over private VPN" \
| sudo tee /var/www/html/index.html
```

On Azure VM:

```
echo "Response from Azure VM over private VPN" \
| sudo tee /var/www/html/index.html
```

20. Test both directions

From EC2 to Azure VM

```
ping -c 4 10.20.1.10
curl --connect-timeout 5 http://10.20.1.10
```

Expected HTTP response: **Response from Azure VM over private VPN**

From Azure VM to EC2

```
ping -c 4 10.10.1.10
curl --connect-timeout 5 http://10.10.1.10
```

Expected HTTP response: **Response from AWS EC2 over private VPN**

SSH (optional)

```
ssh <remote-username>@<remote-private-ip>
```

SSH additionally requires the correct username and an authorized key. Network connectivity alone does not grant login access.

Record your results

Test	Expected result
AWS Tunnel 1	UP
Azure connection	Connected
EC2 → Azure private HTTP	Successful response
Azure VM → EC2 private HTTP	Successful response
Private ping	Replies, if ICMP is permitted
Private SSH	Login, if keys and accounts are configured

Successful HTTP tests in both directions are stronger evidence than a “Connected” gateway screenshot alone.

21. Understand the packet flow

For an EC2 request to **10.20.1.10**:

1. EC2 sends a packet addressed to the Azure VM’s private IP.
2. The AWS subnet route table matches **10.20.0.0/16**.
3. Traffic goes to the AWS Virtual Private Gateway.
4. The gateway encrypts the packet.
5. The encrypted packet travels between the gateway public endpoints.
6. Azure VPN Gateway decrypts it.
7. Azure routes the inner packet to the VM.
8. NSG and host firewall rules determine whether the application receives it.
9. The response returns through the VPN.

The private addresses identify the servers. The public gateway addresses transport the encrypted tunnel packets.

22. Automate the project with Terraform

Use the manual steps above to understand each resource, then express the same design in Terraform.

Suggested project structure

```
aws-azure-vpn/
  versions.tf
  providers.tf
  variables.tf
  aws-network.tf
```

```

azure-network.tf
vpn.tf
compute.tf
outputs.tf
terraform.tfvars.example
.gitignore
README.md

```

Terraform resource mapping

Infrastructure	Terraform resource
AWS VPC	<code>aws_vpc</code>
AWS subnet	<code>aws_subnet</code>
AWS subnet routing	<code>aws_route_table</code> , <code>aws_route_table_association</code> , <code>aws_route</code>
AWS Security Group	<code>aws_security_group</code>
AWS Virtual Private Gateway	<code>aws_vpn_gateway</code>
AWS Customer Gateway	<code>aws_customer_gateway</code>
AWS VPN connection	<code>aws_vpn_connection</code>
AWS VPN static prefix	<code>aws_vpn_connection_route</code>
EC2	<code>aws_instance</code>
Azure resource group	<code>azurerm_resource_group</code>
Azure VNet and subnets	<code>azurerm_virtual_network</code> , <code>azurerm_subnet</code>
Azure gateway public IP	<code>azurerm_public_ip</code>
Azure VPN Gateway	<code>azurerm_virtual_network_gateway</code>
Azure remote endpoint object	<code>azurerm_local_network_gateway</code>
Azure VPN connection	<code>azurerm_virtual_network_gateway_connection</code>
Azure NSG	<code>azurerm_network_security_group</code>
Azure NIC	<code>azurerm_network_interface</code>
Azure Ubuntu VM	<code>azurerm_linux_virtual_machine</code>

For this VNet gateway design, use `azurerm_virtual_network_gateway_connection`. The similarly named `azurerm_vpn_gateway_connection` belongs to a different Azure gateway resource model.

Essential cross-cloud references

These are connection excerpts, not a complete standalone Terraform project. The networking, gateway, compute, and security resources must also be defined.

AWS Customer Gateway references Azure's gateway public IP:

```

resource "aws_customer_gateway" "azure" {
  bgp_asn      = 65515
  ip_address   = azurerm_public_ip.vpn.ip_address
  type        = "ipsec.1"
}

```

AWS VPN references the AWS gateways:

```
resource "aws_vpn_connection" "azure" {
  vpn_gateway_id      = aws_vpn_gateway.this.id
  customer_gateway_id = aws_customer_gateway.azure.id
  type                = "ipsec.1"
  static_routes_only  = true

  # Define tunnel options matching your Azure policy.
}
```

Add Azure's remote network:

```
resource "aws_vpn_connection_route" "azure" {
  vpn_connection_id      = aws_vpn_connection.azure.id
  destination_cidr_block = "10.20.0.0/16"
}
```

Add the EC2 subnet route:

```
resource "aws_route" "azure" {
  route_table_id      = aws_route_table.workload.id
  destination_cidr_block = "10.20.0.0/16"
  gateway_id          = aws_vpn_gateway.this.id
}
```

Azure Local Network Gateway references AWS Tunnel 1:

```
resource "azurerm_local_network_gateway" "aws_tunnel1" {
  name                = "aws-tunnel1-lng"
  location            = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name

  gateway_address = aws_vpn_connection.azure.tunnel1_address
  address_space   = ["10.10.0.0/16"]
}
```

Azure connection references the matching tunnel key:

```
resource "azurerm_virtual_network_gateway_connection" "aws_tunnel1" {
  name                = "azure-to-aws-tunnel1"
  location            = azurerm_resource_group.this.location
  resource_group_name = azurerm_resource_group.this.name

  type                = "IPsec"
  virtual_network_gateway_id = azurerm_virtual_network_gateway.this.id
  local_network_gateway_id  = azurerm_local_network_gateway.aws_tunnel1.id

  shared_key          = aws_vpn_connection.azure.tunnel1_preshared_key
  enable_bgp          = false
  connection_protocol = "IKEv2"

  use_policy_based_traffic_selectors = false

  ipsec_policy {
    ike_encryption = "AES256"
    ike_integrity  = "SHA256"
    dh_group       = "DHGroup14"

    ipsec_encryption = "AES256"
    ipsec_integrity  = "SHA256"
    pfs_group        = "PFS2048"

    sa_lifetime = 3600
    sa_datasize = 102400000
  }
}
```

}

The AWS VPN resource exposes tunnel endpoint and key attributes that can supply these Azure references. Check the resource documentation for your pinned provider version.

Terraform can infer dependencies from the references. If gateway attachment or readiness needs an explicit dependency, add it to the relevant resource.

Run Terraform

After completing all resource files and provider configuration:

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

- Use authenticated AWS and Azure identities; do not embed access keys in the code.
- Pin tested provider versions and commit `.terraform.lock.hcl`.
- If you created resources manually first, either import them into Terraform or use a separate lab environment. Terraform does not automatically adopt existing resources.

Protect the state

Terraform state and saved plans can contain VPN keys and other sensitive values. Example `.gitignore`:

```
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
tfplan
terraform.tfvars
*.pem
*.key
```

Use an encrypted remote backend with restricted access for durable work. Marking an output sensitive hides it from normal display; it does not remove the secret from state.

23. Troubleshoot systematically

Problem	What to check
Tunnel stays DOWN	Gateway public IPs, PSK, IKE version, matching IPsec policy
Tunnel UP, HTTP fails	Routes, SG, NSG, host firewall, Nginx status
Only one direction works	Return route and destination-side firewall rules
Ping fails but HTTP works	ICMP may be blocked; HTTP still proves application connectivity
SSH fails but HTTP works	SSH service, username, key, TCP 22 rules
Azure does not show the AWS route	LNG address space, connection status, route propagation
AWS cannot reach Azure	VPN static prefix and actual EC2 subnet route-table association

Problem	What to check
Small requests work, larger transfers fail	Investigate tunnel MTU, fragmentation, and TCP MSS
Terraform proposes duplicates	Existing resources may need importing
Tunnel 2 remains DOWN	Expected if this first lab configures only Tunnel 1

Useful server checks:

```
ip addr
ip route
sudo systemctl status nginx
sudo ss -lntp
sudo ufw status
```

`ip route` shows the VM's operating-system routes. It does not show the complete managed-cloud routing decision; also inspect AWS route tables and Azure effective routes.